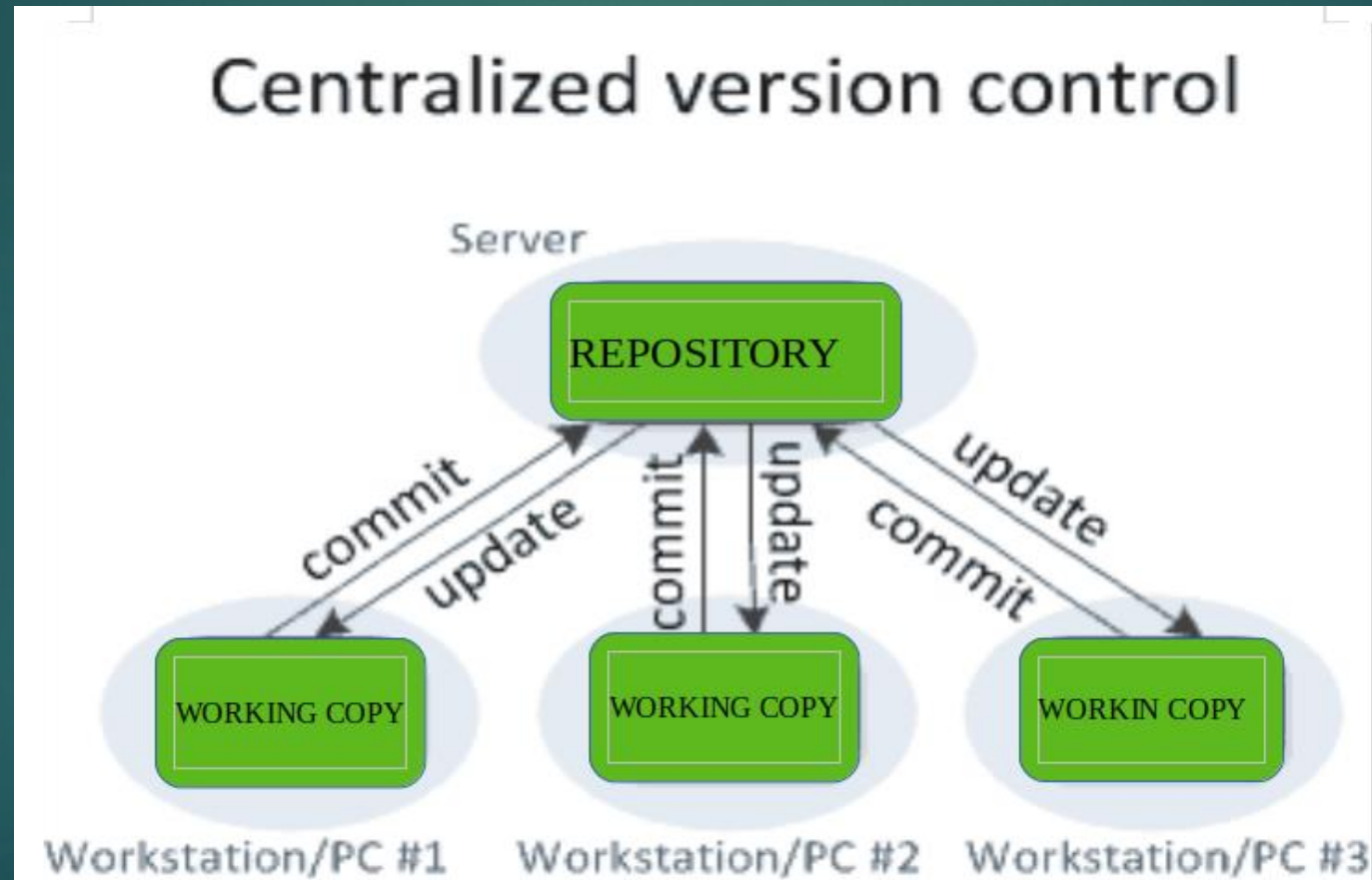


GIT AND GITHUB

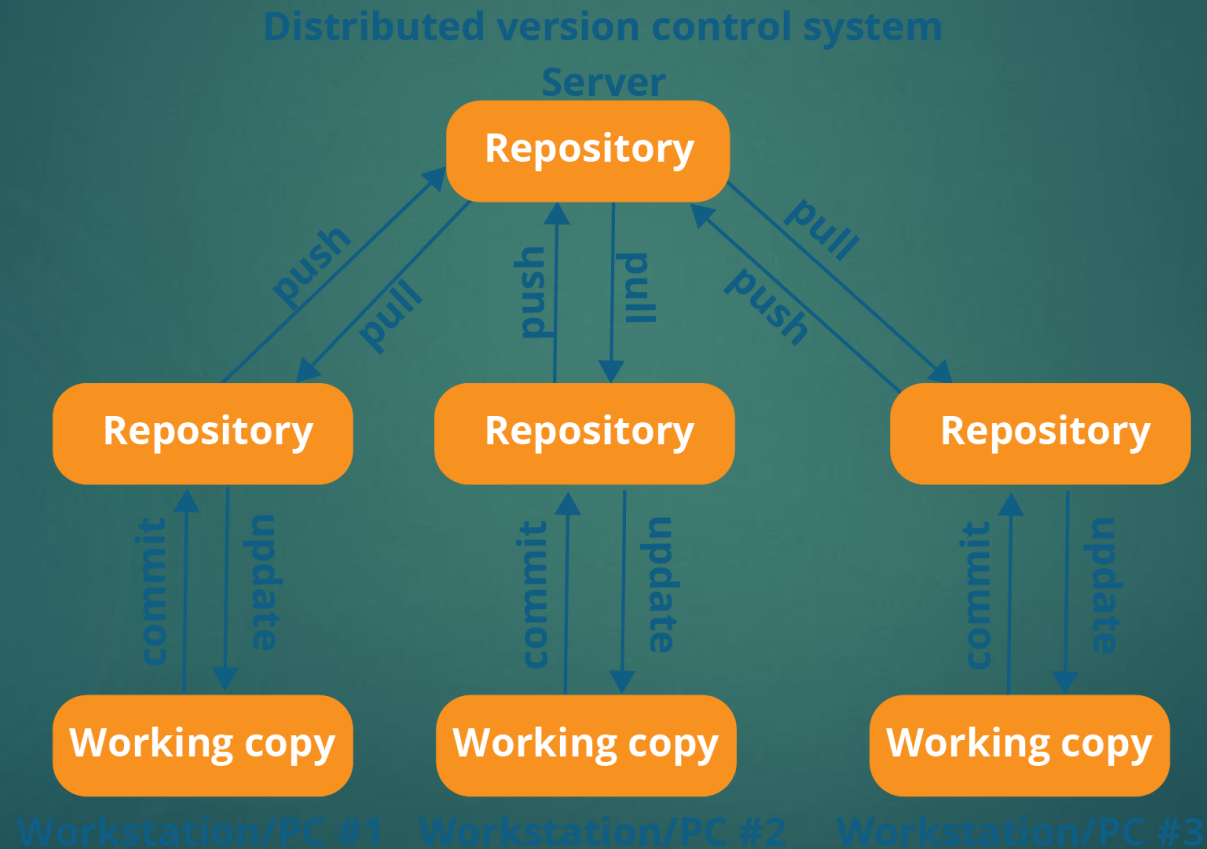
SOURCE CODE

- ▶ WHAT IS SOURCE CODE.?
- ▶ HOW CODE IS HANDELED..?

CENTRAL VERSION CONTROL SYSTEM



DISTRIBUTED VERSION CONTROLLED SYSTEM



WORKING OF GIT



WORKING OF GIT

- ▶ Git has three stages of workflow -**1. Working aria 2. Staging aria 3. Local Repository.**
- ▶ We send data or code from working aria to staging aria by **add command** and staging aria to Local repository by
- ▶ **commit command** and finally send data/code from Local repo to central repo by **push command**.

INSTALLING AND CHECKING GIT ON LINUX

- ▶ `# yum install git -y`
- ▶ `# which git`
- ▶ `User /bin /git`
- ▶ `# git --version`

CONFIGURING GIT ON LINUX

- ▶ # git config --global user.name "LOGAN"
- ▶ # git config --global user.email "LOGAN@gmail.com"
- ▶ # **git config -list**

CREATING A LOCAL REPOSITORY

- ▶ # mkdir **GITLOCAL**
- ▶ # cd GITLOCAL
- ▶ # **git init** (**init command** turn Dir into local Repo)
- ▶ # cat >**GITCLASS** (write inside the [**Dir**] local repo by **cat >** command)

GIT AND GITHUB CLASS.

git status [RUN THIS COMMAND AND NOTE DOWN THE OUTPUT]

GIT ADD FILENAME [TO MOVE A FILE FROM STAGING AREA TO LOCAL REPO]

- 
- ▶ `git commit -m "first commit "` (**m**=message)
 - ▶ `# git status`
 - ▶ `# git log`
 - ▶ `git show <commit-id>`

LINKING LOCAL REPO WITH CENTRAL REPO

- ▶ If want to send this code to my central repository, I have to connect local repo to central repo first, for this action I
- ▶ have to create a new repository (any name) and paste the URL of git repo and execute command as given below
- ▶ **# git remote add origin <https://URL>**
- ▶ **Now local repo has been connected to central repo, for pushing data to central repo execute this command**
- ▶ **# git push -u origin master** (push command local repo to central repo)

- 
- ▶ It will ask for **username and password** of your git hub account, after filling this and you can see all committed data/code inside central repo.

GITIGNORE

- ▶ This command is used to ignore some specific file which we don't want to add & commit.
- ▶ **# vi .gitignore**
- ▶ * CSS **used to ignore particular file**
- ▶ * java
- ▶ Esc+:wq
- ▶ **# git add .gitignore**
- ▶ **# git commit -m** "ignore file format" can use single comma as well
- ▶ **# git status**
- ▶ Nothing to commit, working tree is clean, now create some files in different formats by using **touch command**

- 
- ▶ **# touch** file1.txt file4.java file3.css file5.java file2.txt
 - ▶ **# ls**
 - ▶ **# git status**
 - ▶ File1.txt [only showing 2 untracked files rest three have been ignored]
 - ▶ File2.txt
 - ▶ **# git add FILE NAME**
 - ▶ **# git status**
 - ▶ Now both files have been added and showing us in green color after status command
 - ▶ **File1.txt**
 - ▶ **File2.txt**
 - ▶ **# git commit -m "IGNORE JAVA CSS FILES"**
 - ▶ **# git log**

- 
- ▶ If I want to latest commit, last 2 commits, last-n commits and all commits in one line.
 - ▶ # git log -1
 - ▶ # git log -2
 - ▶ # git log --oneline

GIT BRANCHES:

- ▶ Each task has one separate branches, after done with code other branches merge with master.
- ▶ This concept is useful for parallel development. Master branch is default branch
- ▶ We make branches, one for little features and other one for longer running features.
- ▶ It allows keeps the main master branch free from error.
- ▶ Files created in workspace will be visible in any of the branch workspace, until you commit, once
- ▶ you commit then those files belong to that particular branch

Branches:

- ▶ # git branch
- ▶ ***master**
- ▶ # git branch **branch1**
- ▶ # git branch
- ▶ ***master**
- ▶ Branch1
- ▶ # git **checkout** branch1 (switch to branch
- ▶ branch1)
- ▶ **master**
- ▶ ***Branch1**
- ▶ # git branch **-d** <branchname> (to delete any
- ▶ branch)

Branches Working process:

- ▶ # git checkout **branch1**
- ▶ # **cat >logfile** (create logfile and write anything inside **by >**)
- ▶ Nothings is better that something
- ▶ If you want to add lines or something on this code inside file use command # **cat >>file**, for rewrite use **>**
- ▶ # **ls**
- ▶ branch1 **logfile**
- ▶ # git checkout **master**
- ▶ # **ls**
- ▶ K.txt **logfile**
- ▶ logfile and code is **showing** inside master branch because it hasn't committed with any branch yet.
- ▶ # git commit -m "branch1 first commit"
- ▶ # **git log --oneline**
- ▶ Branch1 first commit
- ▶ # git checkout **master**
- ▶ # **git log --oneline**
- ▶ logfile & code will **not show** inside master branch because that file has been committed with Branch1.

How to Merge Branches: we use pulling mechanism, we can't merge branches of different repositories

- ▶ # git checkout **master**
- ▶ # git merge branchA (to verify the merge)
- ▶ Executed checkout command before merge command means, you wanted to merge any branch with master branch
- ▶ # **git log --one-line**
- ▶ Now you can see **All commits** of both branches which have been merged together
- ▶ # **ls**
- ▶ Now you can see **All files** of both branches which have been merged together.
- ▶ # git **push** origin **master**

How to remove untracked files

- ▶ # git clean -n dry run
- ▶ # git clean -f
- ▶ forcefully

GIT RESET: It is a powerful command that is used to undo local changes to the state of a git repository. It used to undo the add . command.

- ▶ **# cat >logan [you will see the content]**
- ▶ **# git add logan**
- ▶ **# git status**
- ▶ New file:logan (now realized did mistake in data wanted to change)
- ▶ **To reset from staging aria**
- ▶ **# git reset logan**
- ▶ **# git reset** (removed data from staging aria)
- ▶ **# git status**
- ▶ logan

- 
- ▶ **# git add logan**
 - ▶ **# git status**
 - ▶ logan
 - ▶ One branch **master** nothing to commit: working tree clean



GIT REVERT: Revert command helps you to undo the existing commit, it doesn't delete any data instead rather get creates a new commit with the included previous files reverted to the previous stat. So, history moves forward

while the stat of your file moves backward.

- ▶ `# cat >logan`
`I LOVE MY INDIA`

- ▶ `# git add logan`

- ▶ `# git commit -m "logan commit"` after commit I realized did wrong commit

- ▶ `# git log --oneline`

- ▶ Now you can see so many commits copy previous commit id just before the mistake and paste on revert command

- ▶ `# git revert 12h3a8g90[commit id]`



Git Hub Clone: go to existing repo in git Hub copy the URL of central repo and paste with run command of Linux machine.

- ▶ **# git clone** <URL git hub repo>
- ▶ **# git clone** *<https://github.com/zshan227/centralgit2.git>*